FACE RECOGNITION ATTENDANCE SYSTEM

WITH CLASS AND SUBJECT MANAGEMENT

A Project Report

submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering with Specialization in Cyber Security

by

MOULI DEY (TNU2022020200026)
SABUJ BISWAS (TNU2022020100027)
ARPITA PAUL (TNU2022020200031)

Under The Supervision of
Trisha Bera

Department of Computer Science and Engineering
The Neotia University, West Bengal

To

AmbujaNeotia
THE NEOTIA UNIVERSITY
उत्तमम् आत्म प्रदीपाय

JUNE 2026

DECLARATION

I declare that this project report titled **Face Reco** submitted in partial fulfillment of the degree of **B. Tech in (Computer Science and Engineering with Specialization in Cyber Security)** is a record of original work carried out by me under the supervision of **Trisha Bera** and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Mouli Dey

.....

Name: Mouli Dey

UID: TNU2022020200026

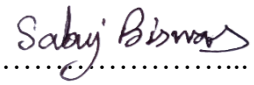
Department: Computer Science & Engineering

24 Parganas (S), West Bengal – 743368

June,2025

DECLARATION

I declare that this project report titled **Face Reco** submitted in partial fulfillment of the degree of **B. Tech in (Computer Science and Engineering with Specialization in Cyber Security)** is a record of original work carried out by me under the supervision of **Trisha Bera** and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.



Name: Sabuj Biswas

UID: TNU2022020200027

Department: Computer Science & Engineering

24 Parganas (S), West Bengal - 743368

June,2025

DECLARATION

I declare that this project report titled **Face Reco** submitted in partial fulfillment of the degree of **B. Tech in (Computer Science and Engineering with Specialization in Cyber Security)** is a record of original work carried out by me under the supervision of **Trisha Bera** and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.



.....

Name: Arpita Paul

UID: TNU2022020200031

Department: Computer Science & Engineering

24 Parganas (S), West Bengal – 743368

June,2025

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to all those who supported us in the successful completion of our project titled *"Face Recognition Attendance System"*.

We are thankful to **Prof. (Dr.) Partha Kumar Mukharjee**, Dean, Department of Computer Science and Engineering, We are thankful to **Dr. Shahnawaz Ansari**, HOD, Department of Computer Science and Engineering, for his encouragement and support throughout the project work.

We express our heartfelt thanks to our project supervisor, **Ms. Trisha Bera**, for her valuable guidance, constant motivation, and timely suggestions, which helped us complete this project successfully.

We also thank all the faculty members and laboratory staff of the Department of Computer Science and Engineering for their cooperation and support.

Finally, we acknowledge the collective hard work and dedication of our team members — **Mouli Dey, Sabuj Biswas, Arpita Paul**. This project is the result of collaborative effort, where each member contributed significantly to the system design, algorithm development, testing procedures, and documentation. The entire experience has strengthened our technical proficiency and emphasized the importance of teamwork and collaboration.

Mouli Dey (TNU2022020200026)

Sabuj Biswas (TNU2022020100027)

Arpita Paul (TNU2022020200031)

ABSTRACT

Biometric processing, computer vision, and machine learning models have transformed legacy identity verification methods within organizational landscapes. This report documents the design and development of an automated Face Recognition Attendance System featuring integrated class and subject management workflows. Developed entirely inside a Python ecosystem, the software aggregates OpenCV architectures for camera-feed feature extraction, utilizes localized SQLite relational structures for data engineering, and relies on Tkinter wrappers for dashboard compilation.

The architecture handles real-time dataset profiling, batch image transformation, and local model caching to flag student attendance records contactlessly. By removing manual roll calls, the engineering minimizes operational overhead, limits proxy inputs, and ensures transaction safety across parallel academic schedules. Functional tests confirm runtime stability under distinct ambient illumination matrices, confirming scalability across enterprise institutional frameworks.

Contents

List of Figures iii

List of Tables iv

1 INTRODUCTION 1
- 1.1 Attendance Management Frameworks . . . 1
- 1.2 Biometric Facial Recognition . . . 1
- 1.3 Problem Statement & Rationale . . . 1
- 1.4 System Objectives . . . 2
- 1.5 Scope of Work . . . 2

2 LITERATURE SURVEY 4
- 2.1 Historical Review of Tracking Architectures . . . 4
- 2.2 Evolution of Computer Vision Paradigms . . . 4
- 2.3 Algorithmic Classifiers & Feature Extraction . . . 4
- 2.4 Recent Advances in Edge Biometrics . . . 4
- 2.5 Robustness Metrics and Edge Vulnerabilities . . . 5
- 2.6 Anti-Spoofing and Secure Environments . . . 5
- 2.7 Enterprise Scale Deployments . . . 5

3 SYSTEM ARCHITECTURE 7
- 3.1 Architectural Blueprint Overview . . . 7
- 3.2 Functional Block Infrastructure . . . 7
- 3.3 Graphical Presentation Subsystem . . . 7
- 3.4 Detection & Classification Pipeline . . . 7
- 3.5 Data Management Subsystem . . . 8
- 3.6 Operational Execution Flow . . . 8

4 DATABASE DESIGN AND IMPLEMENTATION 10
- 4.1 Data Layer Framework . . . 10
- 4.2 Engine Rationalization Parameters . . . 10
- 4.3 Relational Schema Specifications . . . 10
- 4.4 Transactional Processing Logic . . . 10
- 4.5 System Engineering Methodology . . . 11
- 4.6 Directory Tree Organization . . . 11

5 SYSTEM MODULES AND WORKING PRINCIPLE 12
- 5.1 Modular Software Layout . . . 12
- 5.2 Registration Engine . . . 12
- 5.3 Dataset Compilation Engine . . . 12
- 5.4 Classifier Compilation Subsystem . . . 13
- 5.5 Real-time Matching Subsystem . . . 13
- 5.6 Curricular Structuring Engine . . . 14
- 5.7 Unified Operational Logic . . . 14

6 RESULTS, TESTING AND CONCLUSION 16
- 6.1 Testing and QA Protocols . . . 16
- 6.2 System Evaluation Analysis . . . 16
- 6.3 Architectural Performance Gains . . . 17
- 6.4 Structural System Limitations . . . 17

6.5 Scalability Objectives 17
6.6 Project Conclusion 17

7 REFERENCES……....18
8 APPENDIX …….20

List of Figures

1 System Data-Flow Pipeline Layout 3
2 Biometric Pipeline Structural Evolution Chart 5
3 Component Architecture Topology Diagram 8
4 Runtime Execution Flow Model 9
5 Relational Database Schema Layout 11
6 Student Onboarding Component Layout 12
7 Biometric Model Training Pipeline 13
8 Class & Curricular Management Dashboard 14
9 Comprehensive Software Operational Flow Model 15
10 Live Software Dashboard Operation View 16

List of Tables

2 System Vulnerability and Problem Matrix 1
3 Core Target Milestone Specifications 2
4 System Technology Configuration Matrix 2
5 Comparative Mapping of Key Academic Literature 4
6 Application Subsystem Configuration Index 7
7 Functional Component Responsibility Matrix 8
8 Sequential Software Operation Logic 9
9 Data Integration Stack Components 10
10 Student Base Relation Properties 10
11 Software File Tree Layout 11
12 Image Ingestion Variables Matrix 13
13 Real-Time Processing Sequential Pipeline 14
14 Integration Test Suite Execution Summary 16
15 Structural System Benefits Specification 17
16 System Operational Bounds and Constraints 17

ABBREVIATIONS / NOMENCLATURE

Abbreviation	Full Definition / Terminology
AI	Artificial Intelligence
CSV	Comma Separated Values
GUI	Graphical User Interface
ML	Machine Learning
OpenCV	Open Source Computer Vision Library
SQLite	Self-contained SQL Database Engine
UID	Unique Identification Number
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 Attendance Management Frameworks

Tracking student attendance forms a core administrative overhead across educational frameworks. Legacy routines rely heavily on manual registers, sheets, or physical spreadsheets. These practices introduce structural bottlenecks, including processing delays, clerical data entry errors, scale limitations, and proxy vulnerabilities. Transitioning to software-driven automated logging mitigates these vectors by establishing a centralized data pipeline with real-time audit capabilities.

1.2 Biometric Facial Recognition

Facial authentication presents a non-invasive, high-throughput approach to identity verification. The framework maps spatial coordinates from live capture streams, indexing facial arrays against registered templates. This system integrates high-definition video capture feeds with structural classification pipelines to evaluate student identities natively, feeding transactional confirmation entries directly into tracking databases.

1.3 Problem Statement & Rationale

Manual systems fail to scale effectively when tracking dynamic configurations of modern academic schedules. Key friction points are organized systematically in the problem metrics index below.

Table 2: System Vulnerability and Problem Matrix

Reference ID	Operational Risk Category	Systemic Impact Profile
P-01	Operational Latency	High time-to-completion metrics during brief sessions
P-02	Relational Integrity Errors	Mapping faults across complex subject-class matrices
P-03	Data Retention Vulnerability	Critical physical loss risks inside legacy paper registries
P-04	Analytic Inaccessibility	Delayed situational tracking logs for administrative review

Deploying a facial authentication paradigm resolves these complications by introducing automated verification pipelines backed by local relational storage engines.

1.4 System Objectives

The operational expectations defined within the project boundary are structured in the layout table below.

Table 3: Core Target Milestone Specifications

Target ID	Functional Development Parameter
OBJ-01	Construct low-latency, real-time facial matching pipelines
OBJ-02	Establish an automated class, section, and subject indexing subsystem
OBJ-03	Formulate anti-duplicate verification safeguards to isolate double logging
OBJ-04	Provide a graphical dashboard optimized for report extraction

1.5 Scope of Work

The software is engineered as a secure desktop application deployable on edge systems using connected hardware peripherals. The underlying storage framework utilizes isolated relational configurations. Future scaling parameters can transition these local edge components into remote microservices, introducing cloud database arrays and deep-learning engine deployment topologies.

Table 4: System Technology Configuration Matrix

Component Layer	Technical Implementation
Development Language	Python Core Ecosystem
Interface System	Tkinter Layout Wrapper
Vision Engine	OpenCV Framework
Relational Storage	SQLite Database Engine
Matrix Computing	NumPy Array Libraries

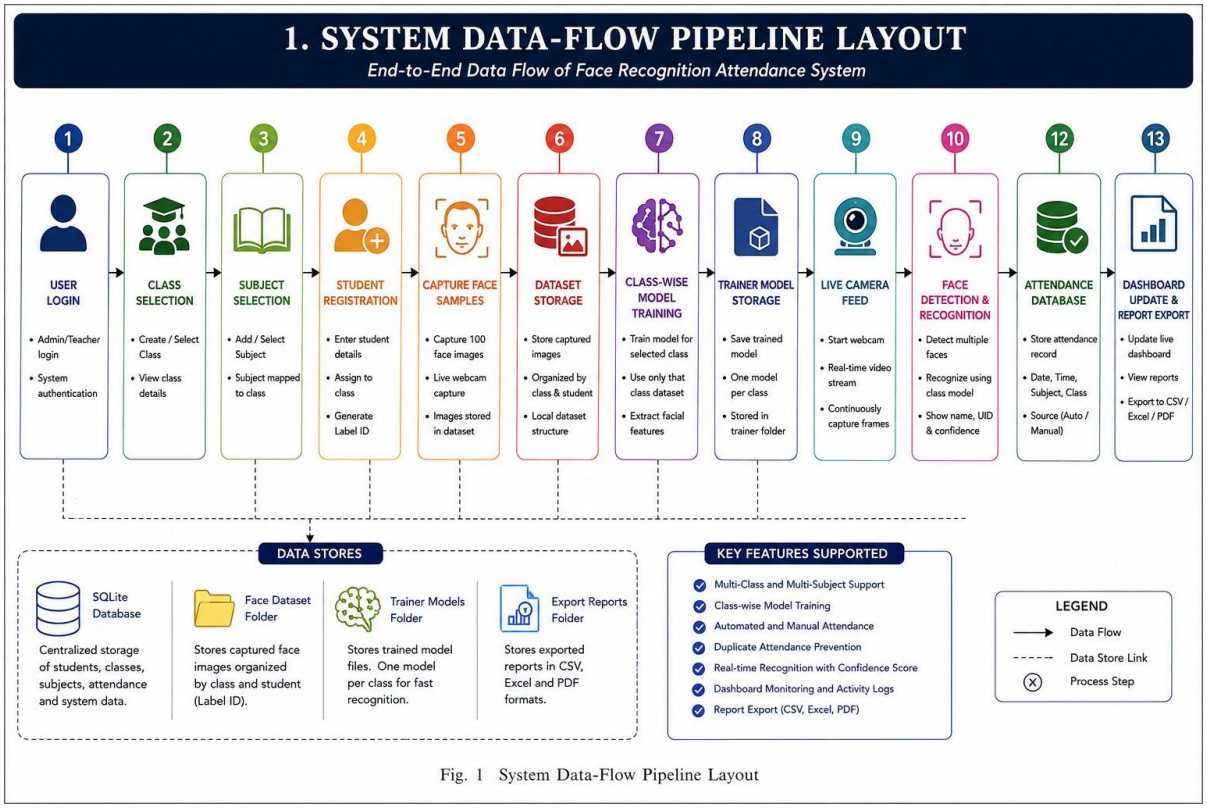


Fig. 1 System Data-Flow Pipeline Layout

Figure 1: System Data-Flow Pipeline Layout

CHAPTER 2

LITERATURE SURVEY

2.1 Historical Review of Tracking Architectures

Traditional institutional operations historically managed structural accounting through pen-and-paper paradigms. Research documentation by Patel and Patel [1] evaluates the structural instability of physical ledgers, defining them as highly error-prone methods. Further inquiries by Ali et al. [5] corroborate these assertions, showing significant drops in system throughput metrics under manual monitoring conditions.

2.2 Evolution of Computer Vision Paradigms

Deploying computer vision workflows to handle verification systems has solved major historical speed-of-processing issues. Analysis by Uma Maheswari et al. [7] tracks the use of algorithmic facial extraction models to handle administrative validation. Fernandes et al. [3] and Lalli et al. [4] expanded this domain by mapping deep-learning classification speeds, demonstrating high precision rates under tightly managed input arrays.

2.3 Algorithmic Classifiers & Feature Extraction

Contemporary recognition frameworks utilize a spectrum of deep networks and feature mapping algorithms. Investigations by Ali et al. [6] present systemic evaluations of LBPH, YOLO, and FaceNet topologies, documenting their capability to isolate identification signatures across dynamic environmental conditions while actively reducing structural fraud loops.

Table 5: Comparative Mapping of Key Academic Literature

Investigator(s)	Year	Methodology	Core Structural Contribution
Patel & Patel	2015	OpenCV Baseline	Real-time classroom deployment validation.
Fernandes et al.	2024	YOLO11 + FaceNet	Deep classification under variant poses.
Anshari et al.	2021	Meta-Analysis	Systemic review of performance constraints.
Khan et al.	2020	OpenCV + API	Cloud hybrid communication validation.
Hosen et al.	2023	Secure Alerts	Anti-spoofing logic implementation.

2.4 Recent Advances in Edge Biometrics

Anshari et al. [11] provide comprehensive analysis proving that automated biometrics reduce system tracking overhead. Sawhney et al. [12] implemented edge OpenCV architectures to con-

firm multi-point validation accuracy in classrooms, while Siswanto et al. [13] proved biometric accuracy profiles outperform hardware RFID systems.

2.5 Robustness Metrics and Edge Vulnerabilities

Deploying deep topologies under fluctuating operational states requires advanced lighting and input compensation. Arsenovic et al. [14] demonstrate that complex models resolve input variations effectively, while Khan et al. [15] explore API processing models to maximize throughput efficiency.

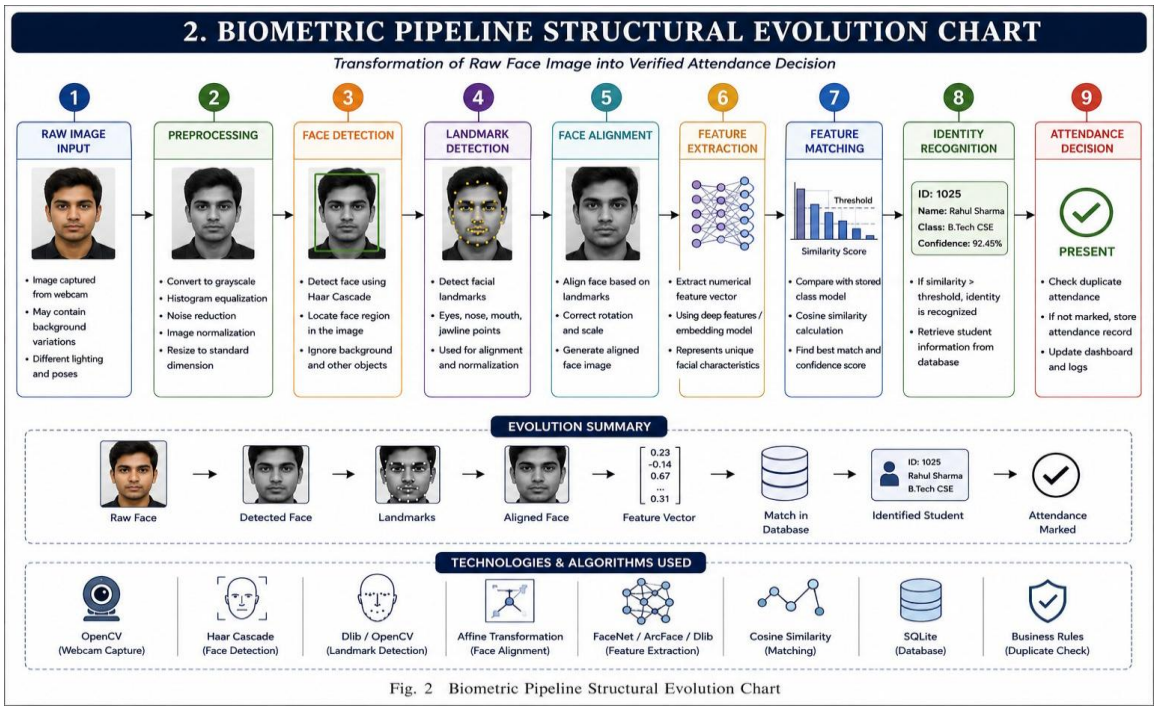


Fig. 2 Biometric Pipeline Structural Evolution Chart

Figure 2: Biometric Pipeline Structural Evolution Chart

2.6 Anti-Spoofing and Secure Environments

Security and spoof protection remain critical priorities for biometric verification loops. Hosen et al. [16] integrate defensive presentation verification to identify falsified media presentations. Saraswat et al. [19] examine the operational deployment of contactless biometrics under public health restrictions, and Salam et al. [20] introduce structural liveness metrics to isolate attack vectors.

2.7 Enterprise Scale Deployments

Expanding biometric systems beyond academic institutions involves adapting processing rules to handle corporate infrastructures. Nguyen-Tat et al. [18] map vision processing layers to

handle enterprise human resource tools. Vardhan et al. [17] detail core edge complications including image scaling limits and privacy-by-design factors, setting foundational paths for future development.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 Architectural Blueprint Overview

The software leverages an independent, modular architecture engineered to decouple the presentation engine from backend database transactions and biometric computations. This structured isolation ensures systemic fault containment and maximizes runtime resource management.

3.2 Functional Block Infrastructure

The structural configurations relies on distinct processing layers configured systematically inside the modular components index table below.

Table 6: Application Subsystem Configuration Index

Module ID	Structural Block Identifier	Engineering Functional Domain
MOD-01	Graphical Presentation Layer	Interfaces, updates layouts, and captures user clicks
MOD-02	Face Capture Subsystem	Connects webcam feeds and maps raw frames
MOD-03	Feature Classification Pipeline	Extracts metrics and builds face geometry maps
MOD-04	Transaction Validation Module	Tracks logs and runs integrity operations
MOD-05	Relational Storage Adapter	Writes log paths and processes SQLite scripts
MOD-06	Academic Matrix Subsystem	Pairs specific targets across class profiles

3.3 Graphical Presentation Subsystem

Constructed upon Tkinter primitives, the UI handles state rendering without demanding significant computational overhead. This engine manages operational modules for student onboarding, dataset compilation, matrix indexing, and diagnostic checking dashboards.

3.4 Detection & Classification Pipeline

The system accesses local hardware camera channels via OpenCV capture pipelines. The classification pipeline parses spatial features from live streams, transforming raw color channels to grayscale frames, normalizing contrast, and outputting target pixel matrices to the verification subsystem.

Table 7: Functional Component Responsibility Matrix

Functional Component	Operational Lifecycle Domain
GUI Layout Manager	Presentation Interface Processing
Matching Engine	Template Matching and Vector Comparison
Storage Layer	Relational Transaction Processing
Validation Pipeline	Anti-Duplicate Parsing Engine
Module Matrix Subsystem	Curricular Structure Definition

3.5 Data Management Subsystem

The storage layout organizes indexing structures, validation parameters, and student records into structural tables. Employing localized transactional engines limits system deployment dependencies while optimizing I/O execution profiles.

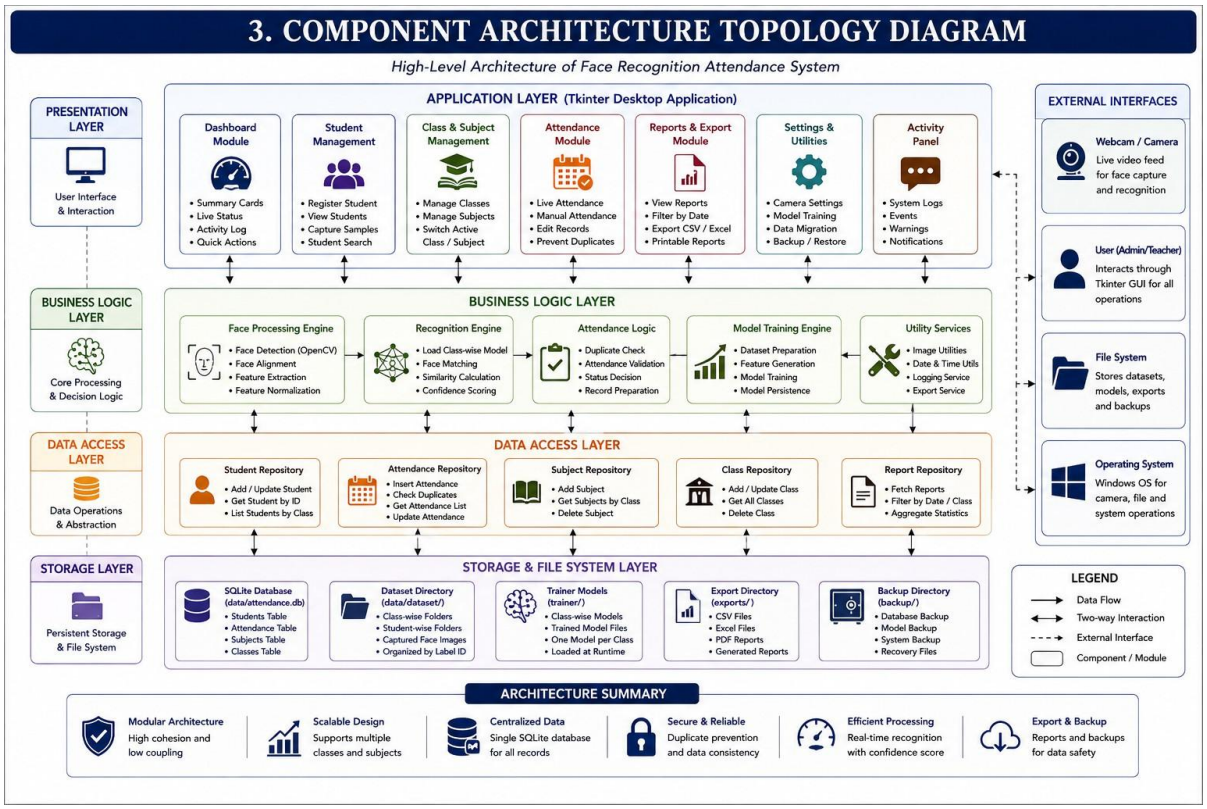


Figure 3: Component Architecture Topology Diagram

3.6 Operational Execution Flow

The application lifecycle runs step-by-step through successive computing cycles, organized in the logical execution table below.

Table 8: Sequential Software Operation Logic

Process Order	Target System Lifecycle Activity
Step 01	Administrator authorization check and main user page initialization
Step 02	Local profile onboarding and initial photo compilation initialization
Step 03	Processing engine model extraction and binary cache serialization
Step 04	Background camera camera device interface synchronization
Step 05	Real-time frame extraction and face template array matching
Step 06	Entry uniqueness assessment via anti-proxy checking flags
Step 07	Relational asset modification commits written to SQLite files
Step 08	Generation of structural compliance analytics as standard CSV output

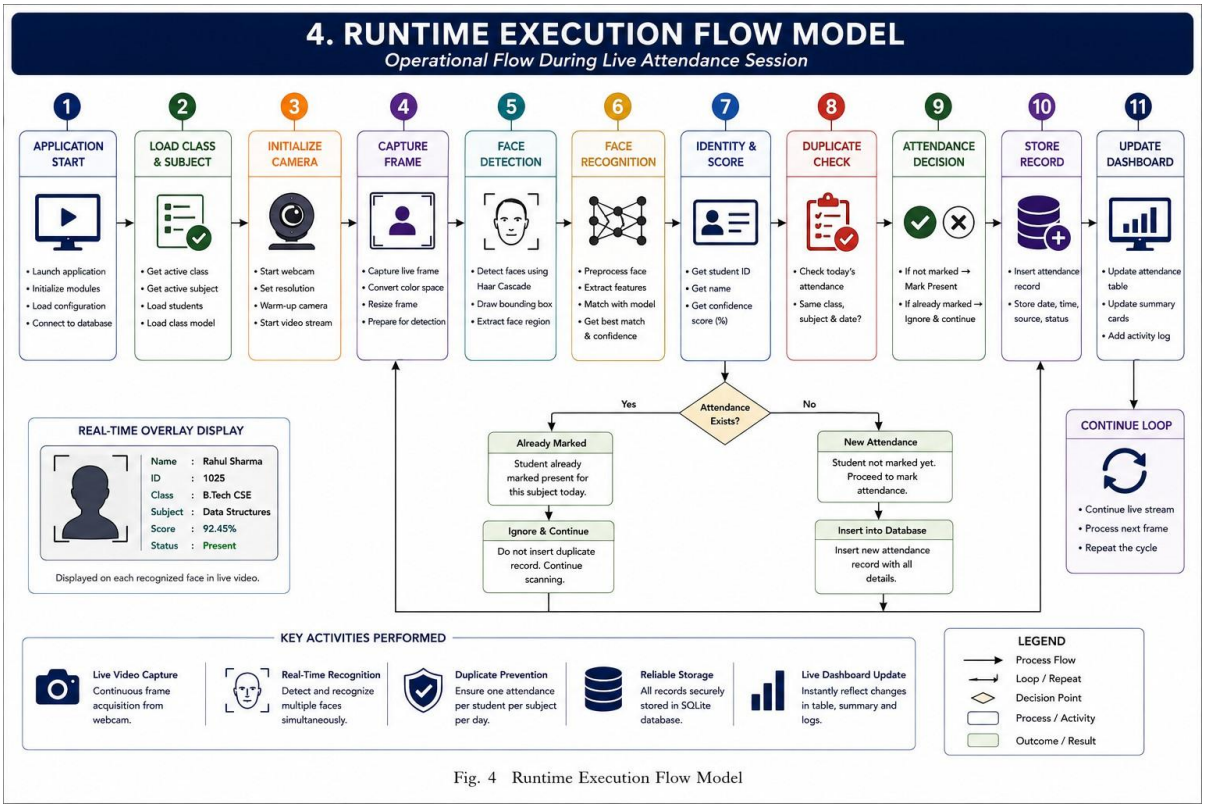


Fig. 4 Runtime Execution Flow Model

Figure 4: Runtime Execution Flow Model

CHAPTER 4

DATABASE DESIGN AND IMPLEMENTATION

4.1 Data Layer Framework

The underlying relational database layout provides structural normalization, fast table joints, and precise transactional data processing. The system architecture prevents anomaly creation across dynamic student, curricular, and attendance tables.

4.2 Engine Rationalization Parameters

SQLite selected as the embedded storage system because it operates serverlessly, minimizes system deployment configurations, and executes relational transactions directly into independent binary files on local storage hardware.

Table 9: Data Integration Stack Components

Infrastructure Component	**Functional Application**
Storage Engine	Embedded Relational Storage File
Interface Wrapper	Natively Compiled Driver Ecosystem
Query Synthesizer	Standard Relational Script Routines

4.3 Relational Schema Specifications

Data storage components are organized into dedicated relation structures: Student Profiles Table, Log Transactions Table, Subject Catalog Table, Class Layout Table, and Dataset Index Table.

Table 10: Student Base Relation Properties

Attribute Identifier	**Data Primitive**	**Structural Constraints**
Student_ID	INTEGER	PRIMARY KEY, NOT NULL
Student_Name	TEXT	VARCHAR(100), NOT NULL
Department	TEXT	VARCHAR(50)
Class	TEXT	VARCHAR(20)
Email	TEXT	UNIQUE, VARCHAR(100)

4.4 Transactional Processing Logic

Following successful pattern matching by the biometric pipeline, the application generates structured SQL inserts. The processing layer validates entity states against the primary tar-

get index, appending timestamp data, module keys, and verification status fields to prevent log duplicates.

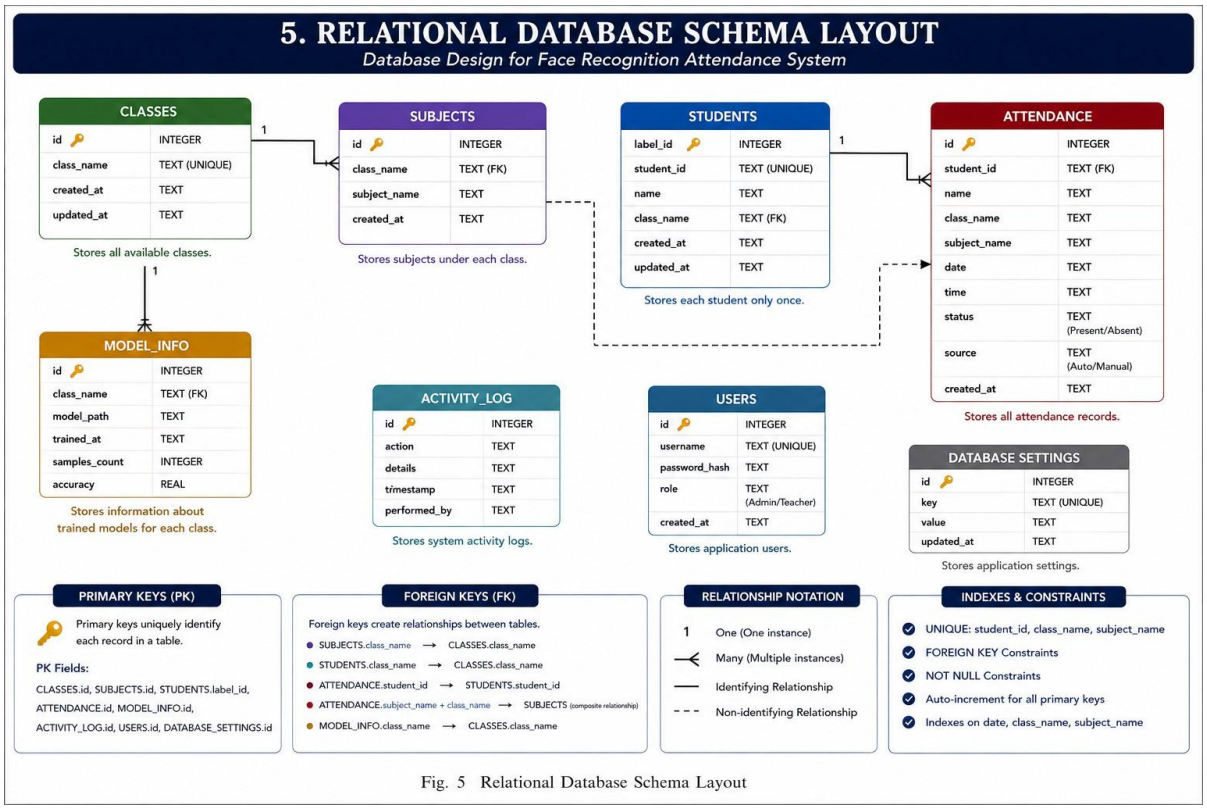


Fig. 5 Relational Database Schema Layout

Figure 5: Relational Database Schema Layout

4.5 System Engineering Methodology

System assembly uses Python scripting languages, integrating GUI wrappers, computer vision frameworks, and database drivers to orchestrate data processing pipelines uniformly.

4.6 Directory Tree Organization

The software layout uses a modular directory configuration to isolate processing libraries from database outputs and media components.

Table 11: Software File Tree Layout

File / Folder Path	Functional Allocation
main.py	Root Process Bootstrapper
student.py	Profile Registration and Ingestion Module
attendance.py	Live Tracking and Matching Logic
dataset/	Image Template Binary Files Repository
database.db	Relational SQLite Target File

CHAPTER 5

SYSTEM MODULES AND WORKING PRINCIPLE

5.1 Modular Software Layout

The system breaks down execution parameters into isolated modules that manage independent processes across the tracking lifecycle.

5.2 Registration Engine

The student ingestion interface captures and validates user details, checking primary constraints before committing profiles to database records.

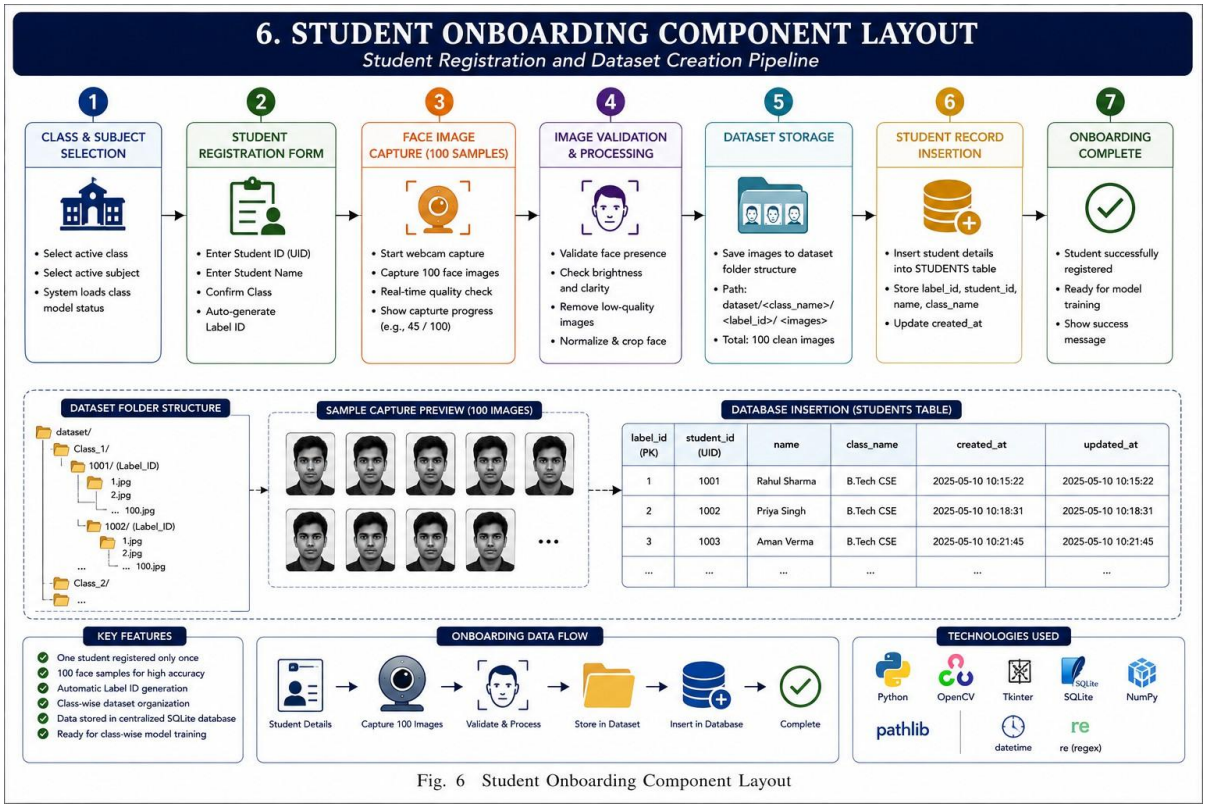


Fig. 6 Student Onboarding Component Layout

Figure 6: Student Onboarding Component Layout

5.3 Dataset Compilation Engine

Following step registration, the capture loop triggers local hardware peripherals to record multiple image frames. Images are converted to grayscale and stored as organized matrices under local directories.

Table 12: Image Ingestion Variables Matrix

Ingestion Variable	Operational State Assignment
Sensor Channel Input	Hardware Webcam Stream
Image Encoding Type	Normalized Grayscale Matrix
Storage Architecture	Local Binary Directory Structure
Optimization Profile	Algorithmic Classifier Reference Matrix

5.4 Classifier Compilation Subsystem

The learning engine reads processed pixel matrices, generating computational feature vectors for each student identity. These generated models are cached as local vector arrays to optimize matching accuracy.

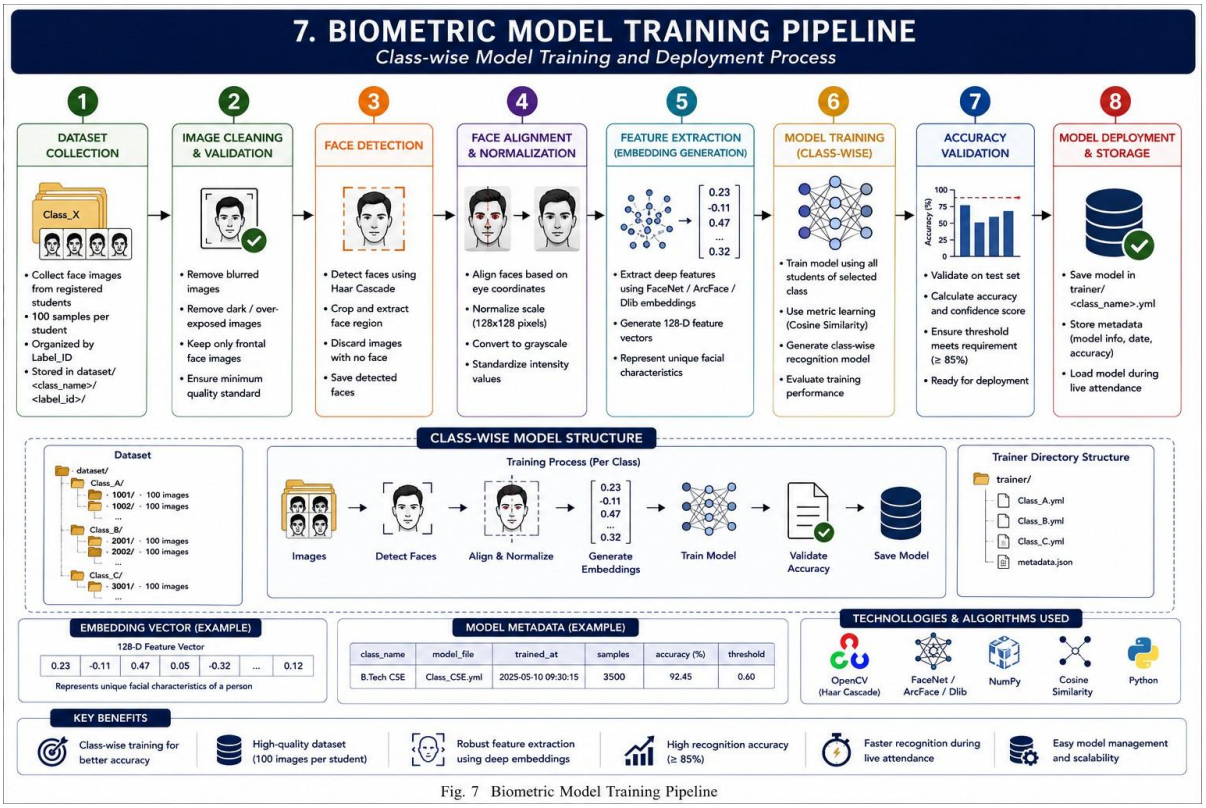


Fig. 7 Biometric Model Training Pipeline

Figure 7: Biometric Model Training Pipeline

5.5 Real-time Matching Subsystem

The execution thread opens live camera channels, runs continuous frame isolation steps, and compares runtime features with cached vectors. Valid matches trigger transaction commits, while anti-duplicate loops check logs to reject double insertions.

Table 13: Real-Time Processing Sequential Pipeline

Sequence Stage	Underlying Computational Process
Phase 1	Video Stream Device Hooking
Phase 2	Face Region Isolation
Phase 3	Vector Matching Execution
Phase 4	Transaction Integrity Evaluation
Phase 5	Database Row Commit

5.6 Curricular Structuring Engine

The subject tracking layout links course profiles across varied academic classes, providing flexible configuration options for complex educational institutions.

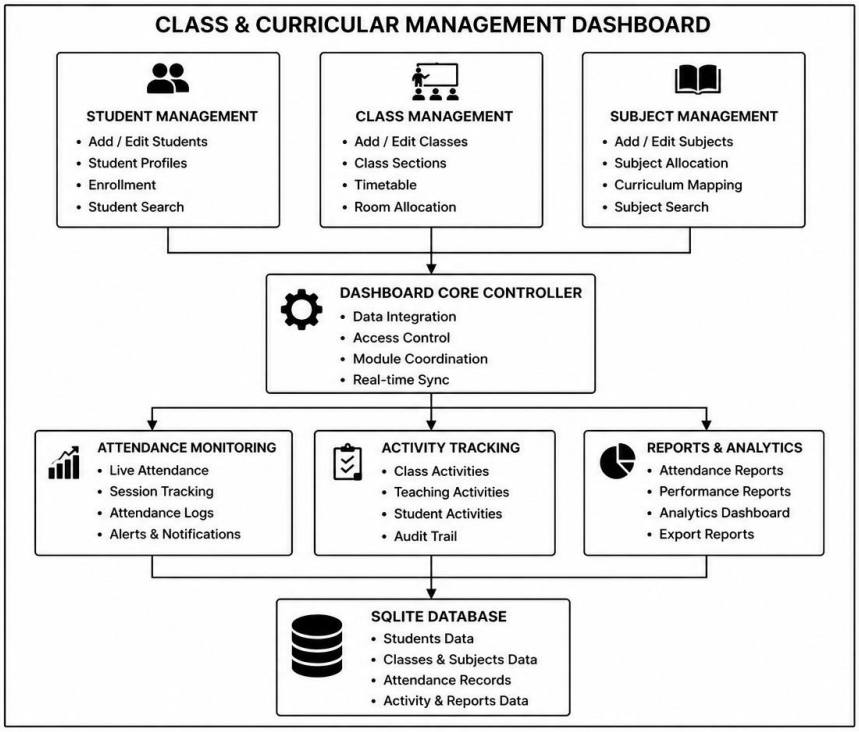


Figure 8: Class and Curricular Management Dashboard Architecture

Figure 8: Class & Curricular Management Dashboard

5.7 Unified Operational Logic

The systemic workflow runs through fixed operational stages: Profile Ingestion, Media Sampling, Vector Generation, Interface Setup, Stream Capture, Feature Matching, Constraint Auditing, Relational Insertion, and Export Generation.

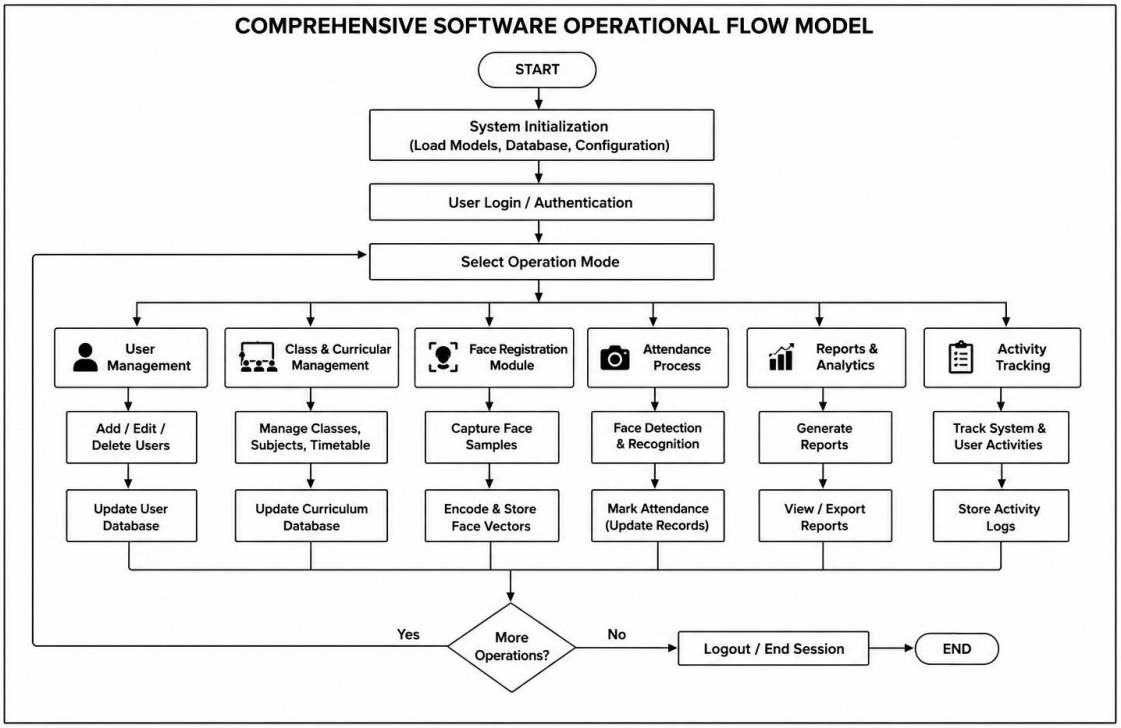


Figure 9: Comprehensive Software Operational Flow Model

Figure 9: Comprehensive Software Operational Flow Model

CHAPTER 6

RESULTS, TESTING

6.1 Testing and QA Protocols

Individual validation runs analyzed component dependencies and overall application stability, testing ingestion flows, model compiling, relational writing, and export routines across multiple execution states.

Table 14: Integration Test Suite Execution Summary

Target Process Boundary	Evaluated Performance Metric	Status
Profile Ingestion Pipeline	Relational Constraint Stability	Passed
Frame Extraction Module	Local File Commit Verification	Passed
Biometric Classifier Engine	Accurate Match Verification	Passed
Relational Data Writing	Index Consistency Audit	Passed
Curricular Layout Interface	Table Join Performance	Passed

6.2 System Evaluation Analysis

The application accurately processes and records facial transactions under target load configurations. Relational logs maintain reference integrity without creating database locking conflicts during continuous tracking cycles.

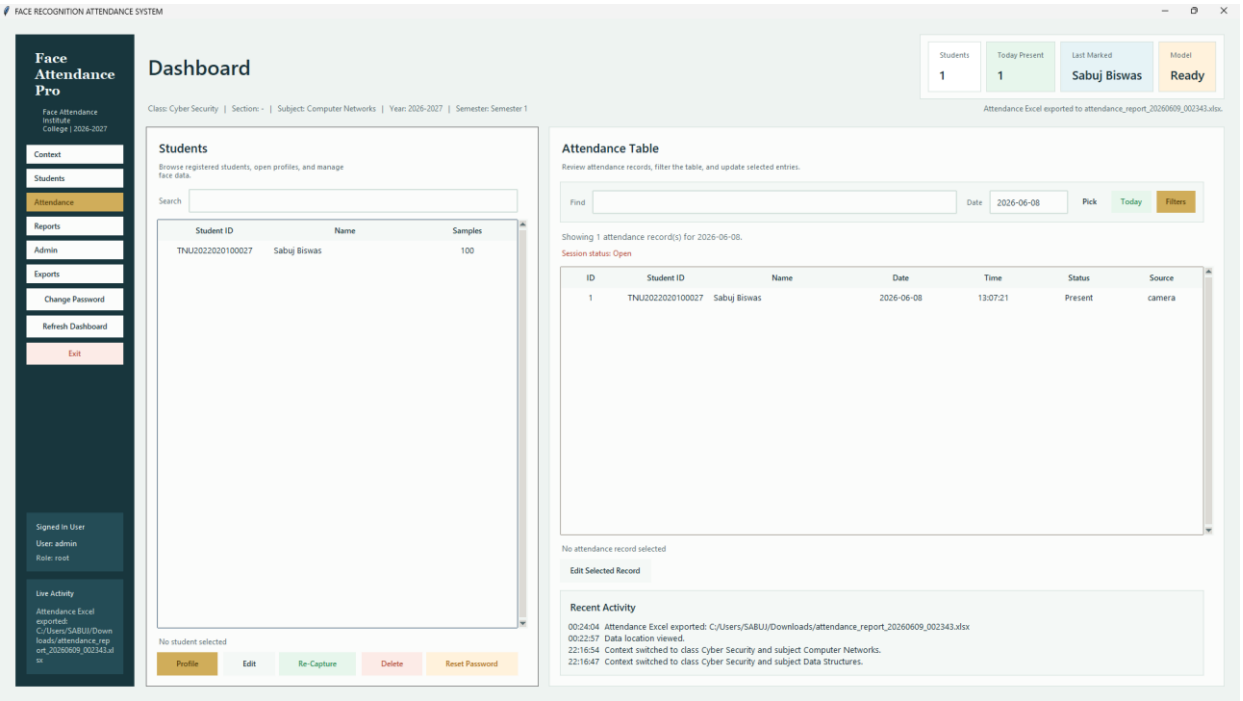


Figure 10: Live Software Dashboard Operation View

6.3 Architectural Performance Gains

The core features delivering optimization properties across performance boundaries are categorized in the evaluation index table below.

Table 15: Structural System Benefits Specification

Target Improvement Area	Architectural Enhancement Impact Profile
Paper Elimination	Drops localized asset operational overhead expenditures
Contactless Interface	Drives automated, non-intrusive logging across hardware devices
Fraud Isolation	Seals historical operational duplicate entries and proxy pathways
Relational Mapping	Syncs student identities uniformly across module records

6.4 Structural System Limitations

Operational barriers identified during performance evaluation runs are systematically mapped in the constraints table below.

Table 16: System Operational Bounds and Constraints

Constraint ID	Identified Performance Boundary Check
LIM-01	Vector calculation accuracy shifts across dark ambient states
LIM-02	Sensor output processing quality relies on external camera setups
LIM-03	Matrix searching lag scales up on expansive database storage sets
LIM-04	Standalone app designs restrict concurrent enterprise admin syncs

6.5 Scalability Objectives

Future development iterations plan to transition the edge storage architecture to centralized cloud clusters, integrate deep learning models to handle varying lighting conditions, and build web-based administrative reporting portals.

6.6 Project Conclusion

The Face Recognition Attendance System with Class and Subject Management successfully merges biometric pattern matching, relational database management, and administrative workflows into a unified, high-performance tracking tool. The resulting system limits administrative overhead, eliminates proxy risks, and presents a stable biometric solution tailored for modern academic frameworks.

REFERENCES

1. H. Patel and M. Patel, ”Smart Attendance System Using Face Recognition in Classroom,” *International Journal of Engineering Research and Technology (IJERT)*, 2015.

2. R. Rafeek, V. S. Anoop, and Z. Akhtar, ”FaceTrack: A Face Recognition-based Real-time Attendance Marking Approach using Haar Cascade and Machine Learning,” in *Artificial Intelligence and Optimization Techniques for Smart Information System Generations*, CRC Press, pp. 258–277.

3. F. Fernandes et al., ”One-Shot Person Re-identification Using YOLO11-Based FaceNet Architecture,” in *International Conference on Communication and Intelligent Systems*, Springer Nature Singapore, 2024.

4. K. Lalli, V. Arifulla, and M. Senbagavalli, ”A Smart Automated Attendance Monitoring System Using Deep Learning Model,” in *2024 International Conference on Computing and Intelligent Reality Technologies (ICCIRT)*, IEEE, pp. 1–4, 2024.

5. A. Ali, R. Chakraborty, and N. Alharbe, Eds., *Artificial Intelligence and Optimization Techniques for Smart Information System Generations*, CRC Press, 2025.

6. A. A. Ali, T. Abd El-Hafeez, and Y. K. Mohany, ”An Accurate System for Face Detection and Recognition,” *Journal of Advances in Mathematics and Computer Science*, vol. 33, no. 3, pp. 1–19, 2019.

7. V. Uma Maheswari, R. Aluvalu, and K. K. Chennam, ”Application of Machine Learning Algorithms for Facial Expression Analysis,” *Machine Learning for Sustainable Development*, vol. 9, pp. 77–96, 2021.

8. A. B. Dutta and D. Ghose, ”Automated Attendance Management System Using Face Recognition Techniques,” *International Journal of Computer Applications (IJCA)*, 2015.

9. H. Zhi and L. S. Liu, ”Face Recognition Based on Genetic Algorithm,” *Journal of Visual Communication and Image Representation*, vol. 58, pp. 495–502, 2019.

10. M. Hassaballah and S. Aly, ”Face Recognition: Challenges, Achievements and Future Directions,” *IET Computer Vision*, vol. 9, no. 4, pp. 614–626, 2015.

11. A. Anshari et al., ”Face Recognition for Identification and Verification in Attendance System: A Systematic Review,” in *2021 IEEE International Conference on Communication, Networks and Satellite (COMNETSAT)*, IEEE, 2021.

12. S. Sawhney et al., ”Real-time Smart Attendance System Using Face Recognition Techniques,” in *2019 9th International Conference on Cloud Computing, Data Science and Engineering (Confluence)*, IEEE, 2019.

13. A. R. S. Siswanto, A. S. Nugroho, and M. Galinium, ”Implementation of Face Recognition Algorithm for Biometrics Based Time Attendance System,” in *2014 International Conference on ICT for Smart Society (ICISS)*, IEEE, 2014.

14. M. Arsenovic et al., ”FaceTime—Deep Learning Based Face Recognition Attendance System,” in *2017 IEEE 15th International Symposium on Intelligent Systems and Informatics (SISY)*, IEEE, 2017.

15. S. Khan, A. Akram, and N. Usman, ”Real Time Automatic Attendance System for Face Recognition Using Face API and OpenCV,” *Wireless Personal Communications*, vol. 113, no. 1, pp. 469–480, 2020.

16. M. A. Hosen et al., ”Face Recognition-based Attendance System with Anti-spoofing, System Alert, and Email Automation,” *Radioelectronic and Computer Systems*, vol. 2, pp. 119–128, 2023.

17. R. Vardhan et al., ”Automated Attendance System Using Facial Recognition: A Comprehensive Review,” in *Challenges and Opportunities for Innovation in India*, pp. 117–122, 2025.

18. B. T. Nguyen-Tat, M. Q. Bui, and V. M. Ngo, ”Automating Attendance Management in Human Resources: A Design Science Approach Using Computer Vision and Facial Recognition,” *International Journal of Information Management Data Insights*, vol. 4, no. 2, p. 100253, 2024.

19. D. Saraswat et al., ”Anti-spoofing-enabled Contactless Attendance Monitoring System in the COVID-19 Pandemic,” *Procedia Computer Science*, vol. 218, pp. 1506–1515, 2023.

20. A. E. U. Salam, A. T. K. P. Tarra, and D. Utamidewi, ”Automatic Attendance System Using Silent Face Anti-Spoofing to Detect Spoof on Face Recognition,” in *2025 International Conference on Smart Computing, IoT and Machine Learning (SIML)*, IEEE, 2025.

Appendix

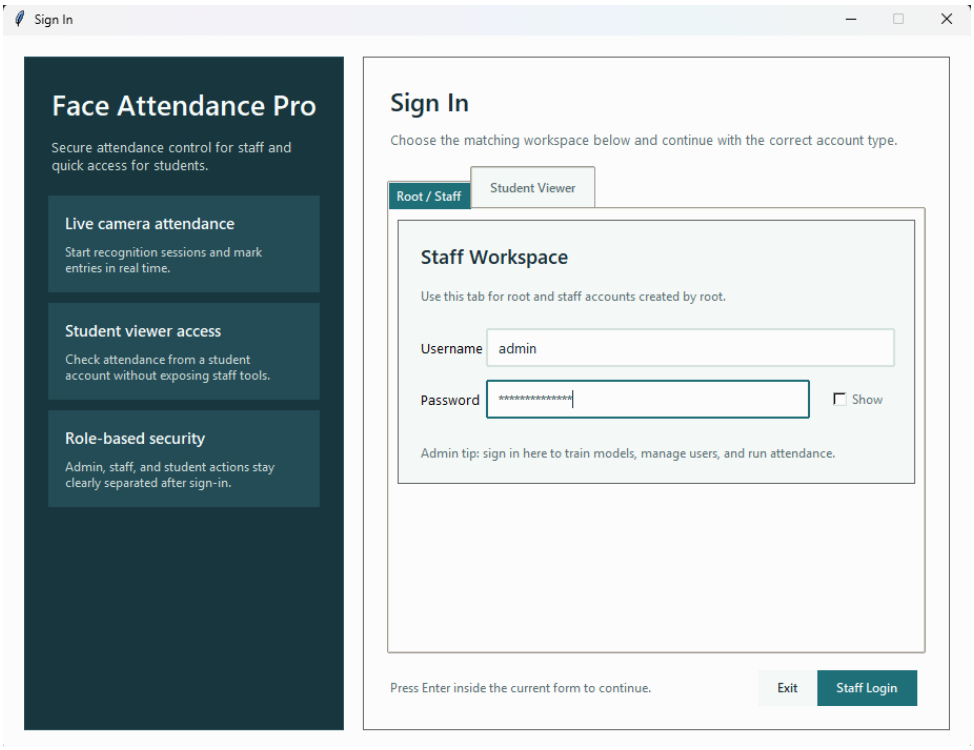


Figure 1 – Application Login Screen

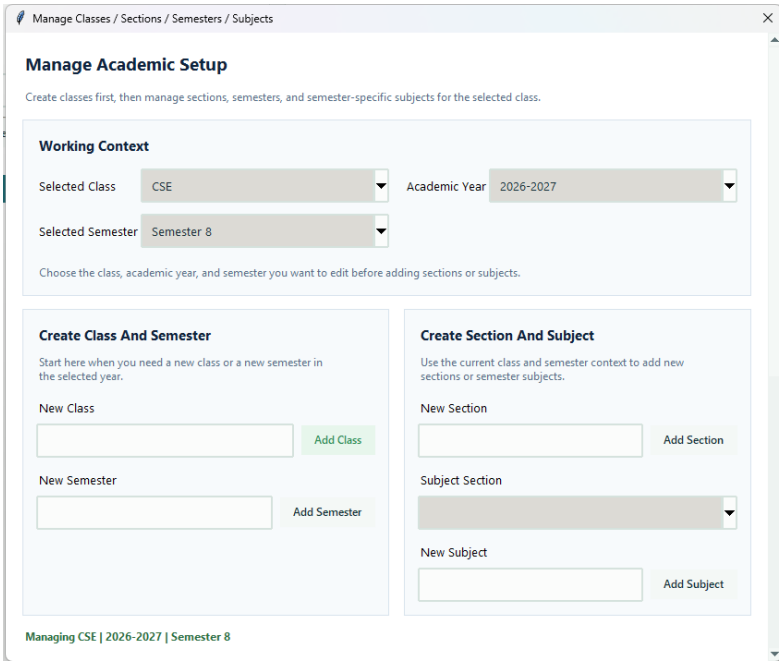


Figure 2 – Manage Academic Set-up

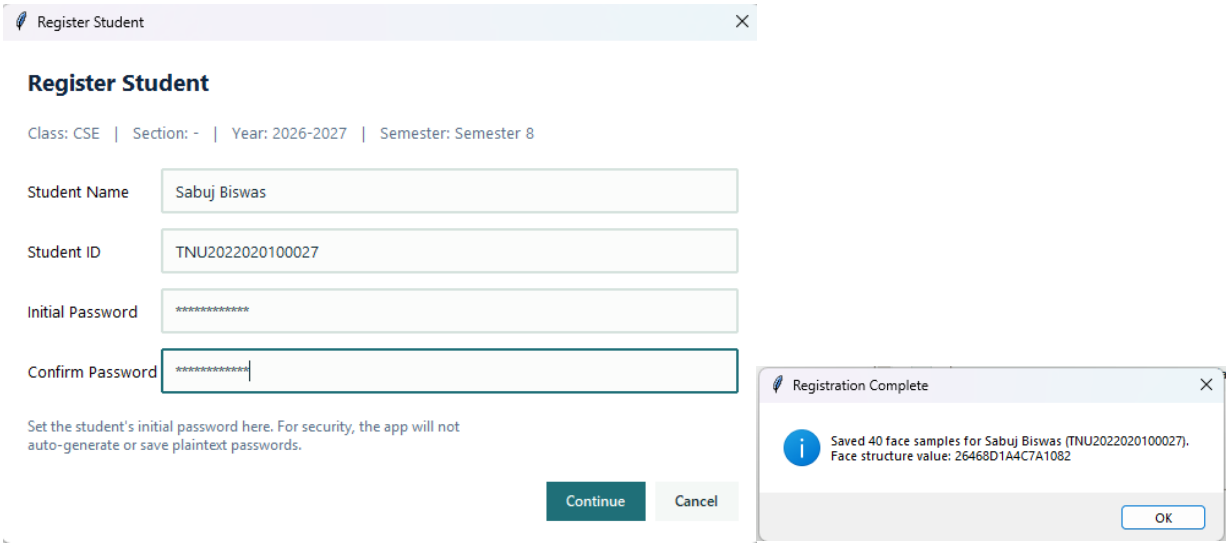


Figure 3 – Register New Students

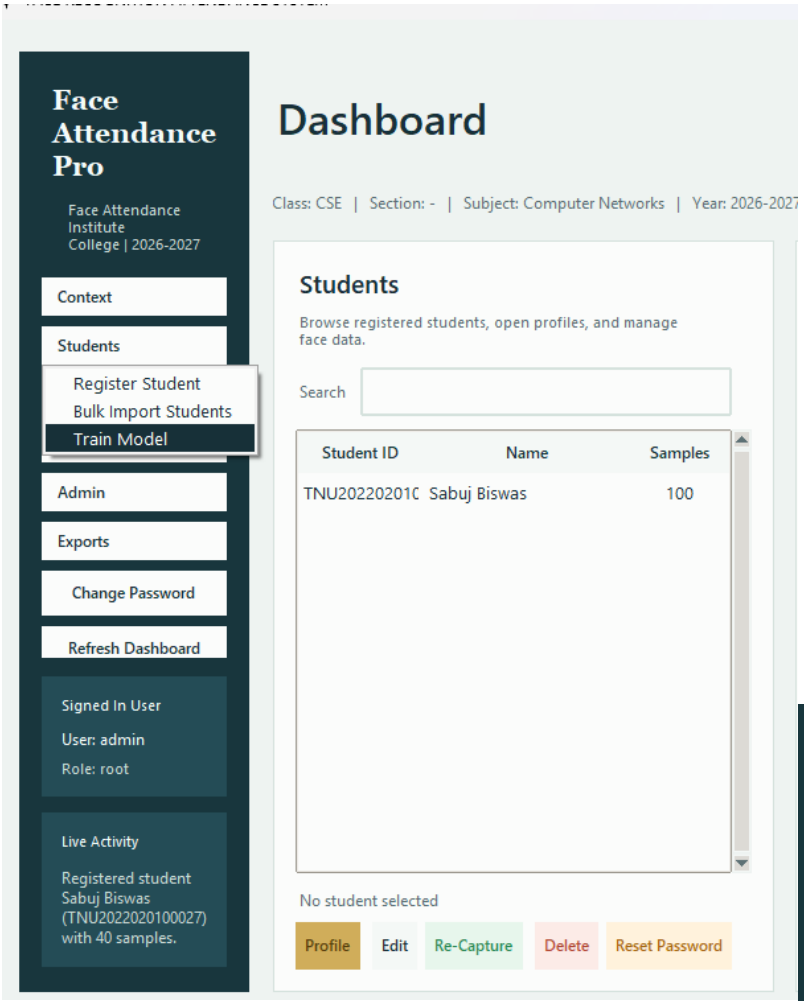
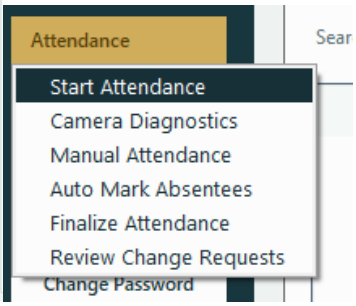


Figure 4 – Train Model with New Students



Start Attendance Button

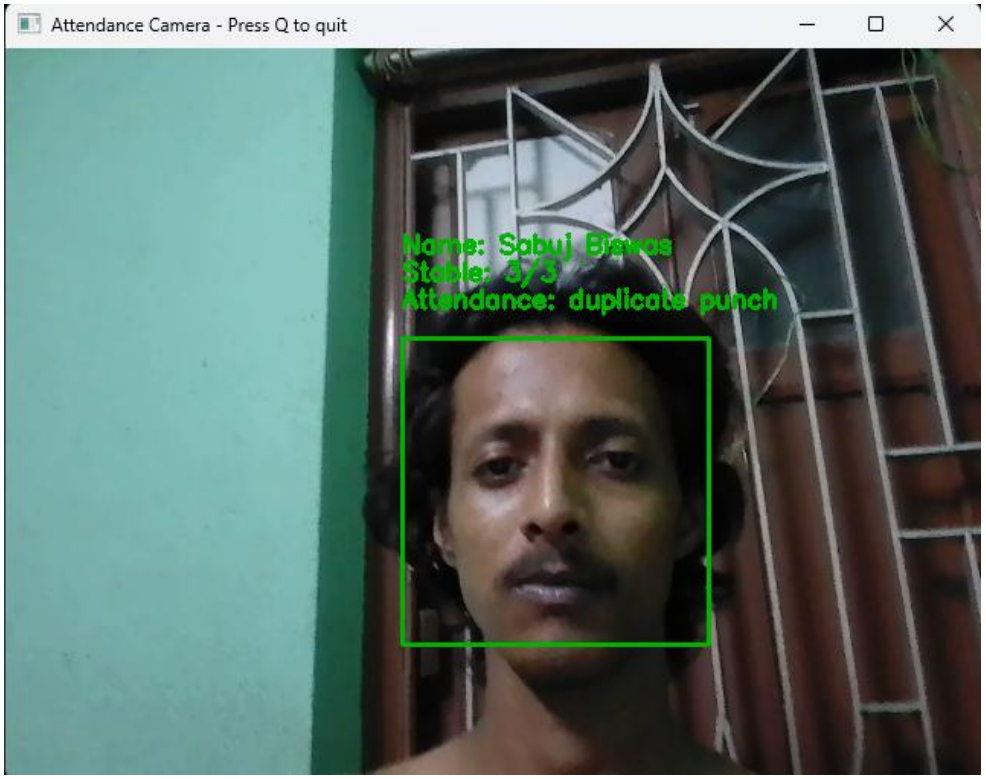


Figure 5 – Live Attendance Mark With Identifying Duplicate Punch

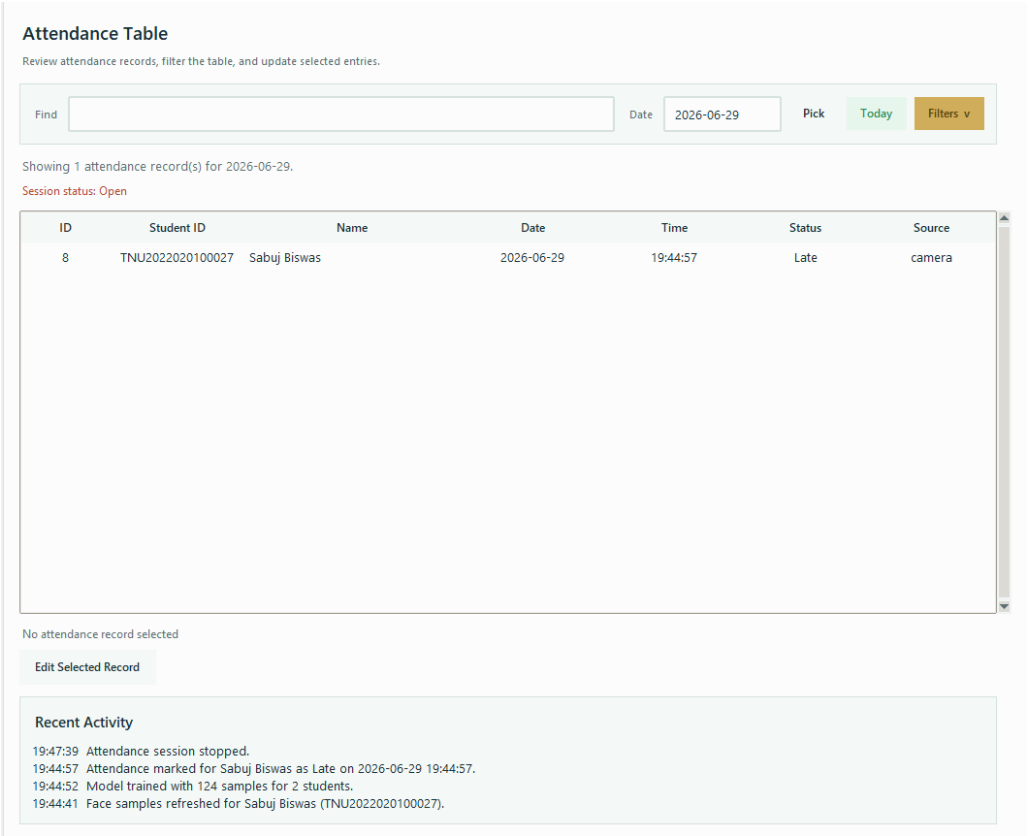


Figure 6 – Attendances Mark in Live Database SQLITE3

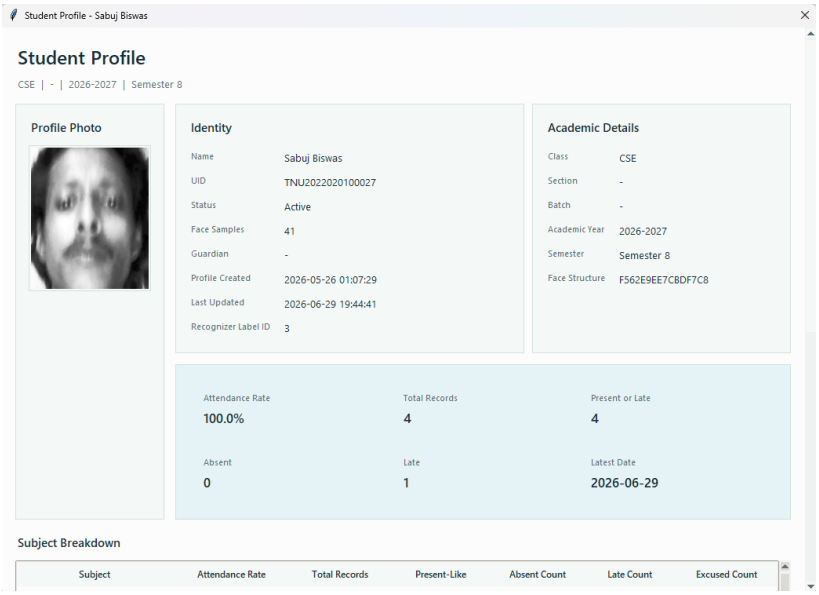


Figure 7 – Student Profiles

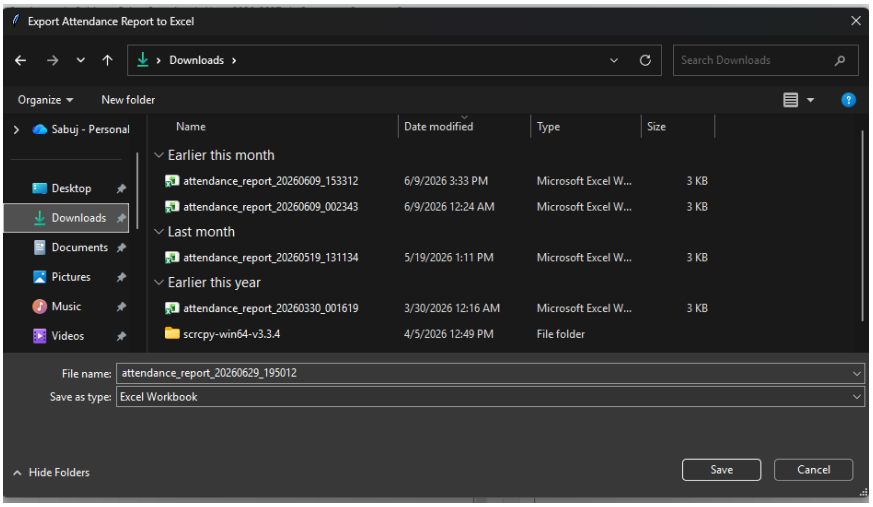


Figure 8 – Exports Attendance to Excel

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	id	student_id	name	class_name	section_name	academic_ye	semester	subject_name	date	time	status	source	
2	8	TNU2022020100027	Sabuj Biswas	CSE		2026-2027	Semester 8	Cyber Security	2026-06-29	19:44:57	Late	camera	
3													
4													

Figure 9 – Excel Data Format

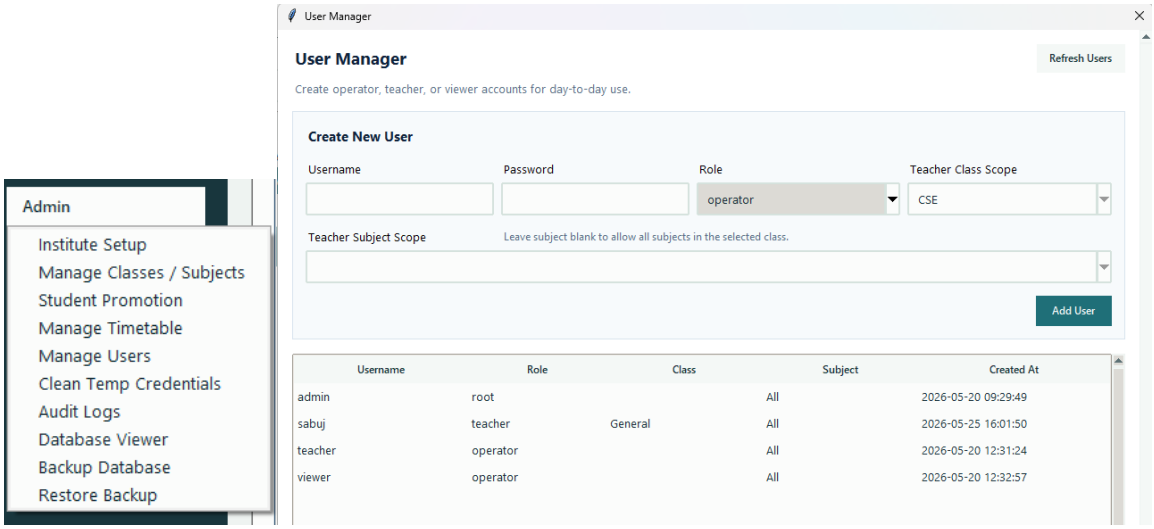


Figure 10 – Admin Features and User Manager Section

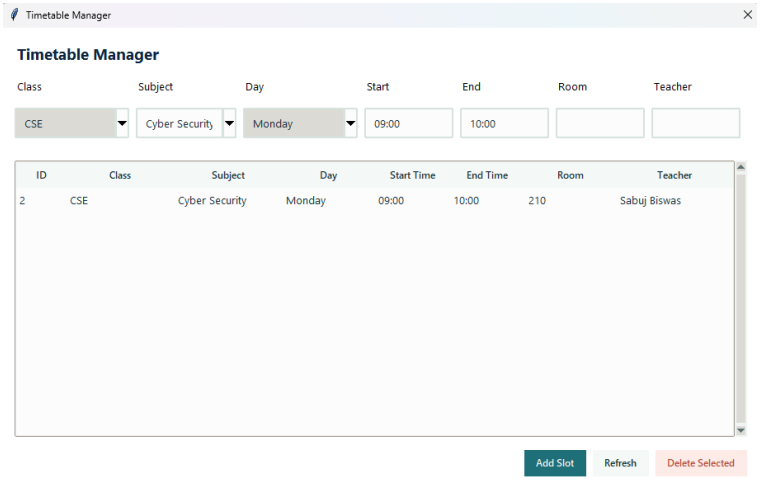


Figure 11 – Time Tables Managers

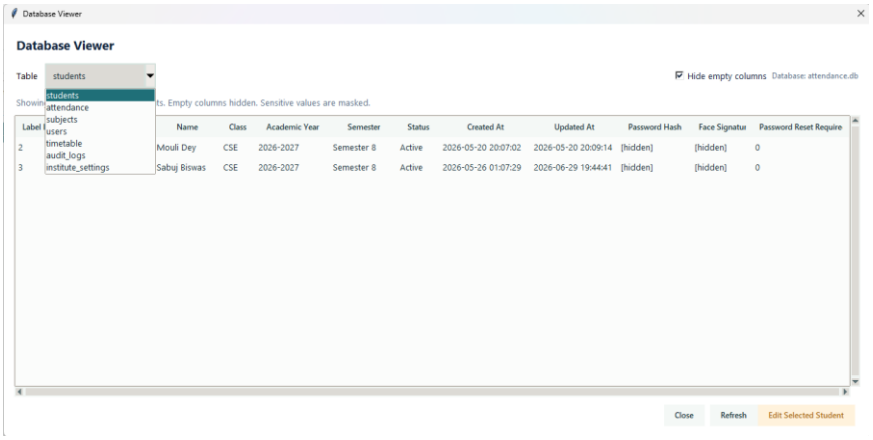


Figure 12 – Database View in The Application Also

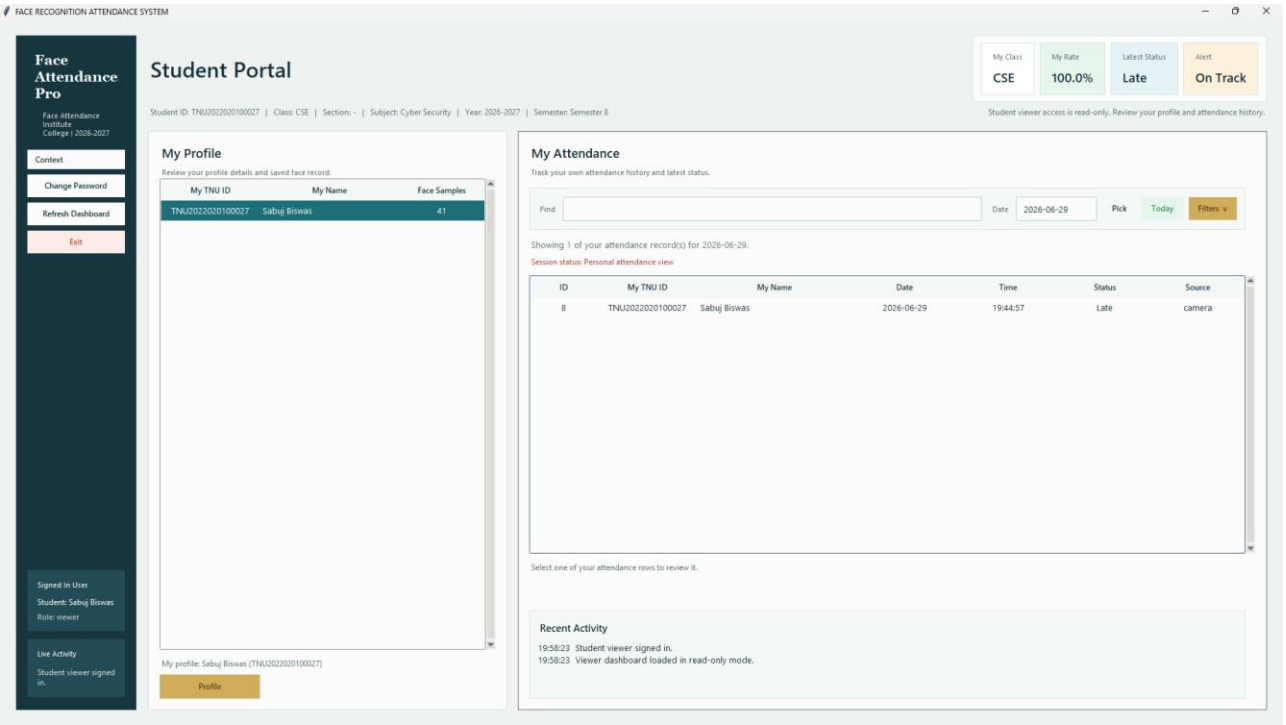


Figure 13 – Student Portales